VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 11: Node JS basics

Branch/ Class: B.Tech/M.Tech **Date:**
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE
Student name: **Reg. no.:**

Exercise 1: Develop a simple Node.js application that creates a web server and responds to client requests. The application should demonstrate how Node.js handles HTTP requests and responses without using any external frameworks.

Functional Requirements

- Create a Node.js application using Node.js runtime environment
- Import required modules using the require() function in Node.js
- Use the built-in http module in Node.js to create a server
- Define a server using the createServer() method
- Handle incoming client requests using request-response callback function
- Send a response to the client using the response object methods (write, end)
- Set appropriate response headers using response.setHeader()
- Run the server on a specific port using the listen() method
- Display server status in the console using console logging in Node.js
- Ensure the server responds correctly when accessed through a browser

VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 11: Node JS basics

Branch/ Class: B.Tech/M.Tech **Date:**
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE
Student name: **Reg. no.:**

Lab_11 > ex-1 > server.js > ...

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "text/plain");

  res.write("Hello from Node.js Server\n");
  res.write("Request received successfully!");

  res.end();
});

const PORT = 3000;

server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

```
PS C:\Users\SYS\Desktop\Web_tecnologies_lab_work\Lab_11> cd ex-1
PS C:\Users\SYS\Desktop\Web_tecnologies_lab_work\Lab_11\ex-1> node server.js
Server running at http://localhost:3000
```

localhost:3000

```
Hello from Node.js Server
Request received successfully!
```

VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 11: Node JS basics

Branch/ Class: B.Tech/M.Tech **Date:**
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE
Student name: **Reg. no.:**

Exercise 2: Develop a Node.js application that performs file operations such as creating, reading, writing, and deleting files. This exercise helps understand how Node.js interacts with the file system.

Functional Requirements

- Create a Node.js application to perform file handling using Node.js File System (fs) module
- Import the file system module using the require() function
- Create a new file using fs.writeFile() method
- Read the contents of a file using fs.readFile() method
- Append data to an existing file using fs.appendFile() method
- Delete a file using fs.unlink() method
- Handle asynchronous operations using callback functions in Node.js
- Manage errors during file operations using error-first callback handling
- Display file operation results in the console using console logging
- Ensure proper execution flow of file operations using asynchronous execution model

VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 11: Node JS basics

Branch/ Class: B.Tech/M.Tech **Date:**
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE
Student name: **Reg. no.:**

Lab_11 > ex-2 > file.js > ...

```javascript
const fs = require("fs");

// 1. Create / Write file
fs.writeFile("test.txt", "Hello Node.js", (err) => {
  if (err) throw err;
  console.log("File created and data written");

  // 2. Append data
  fs.appendFile("test.txt", "\nAppended text", (err) => {
    if (err) throw err;
    console.log("Data appended");

    // 3. Read file
    fs.readFile("test.txt", "utf8", (err, data) => {
      if (err) throw err;
      console.log("File content:");
      console.log(data);

      // 4. Delete file
      fs.unlink("test.txt", (err) => {
        if (err) throw err;
        console.log("File deleted");
      });
    });
  });
});
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\SYS\Desktop\Web_tecnologies_lab_work> cd Lab_11/ex-2
PS C:\Users\SYS\Desktop\Web_tecnologies_lab_work\Lab_11\ex-2> node file.js
File created and data written
Data appended
File content:
Hello Node.js
Appended text
File deleted
PS C:\Users\SYS\Desktop\Web_tecnologies_lab_work\Lab_11\ex-2>
```

VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 11: Node JS basics

Branch/ Class: B.Tech/M.Tech **Date:**
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE
Student name: **Reg. no.:**

Exercise 3: Develop a Node.js application that demonstrates event-driven programming using events and listeners. The application should trigger custom events and respond accordingly.

Functional Requirements

- Create a Node.js application demonstrating events using EventEmitter class
- Import the events module using the require() function
- Create an event emitter object using EventEmitter instance
- Define custom events using the emit() method
- Register event listeners using the on() method
- Execute callback functions when events are triggered using event handling mechanism
- Pass data through events using arguments in emit()
- Handle multiple listeners for a single event using multiple event subscriptions
- Display event responses in the console using console logging
- Demonstrate asynchronous behavior using event-driven architecture in Node.js

VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 11: Node JS basics

Branch/ Class: B.Tech/M.Tech **Date:**
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE
Student name: **Reg. no.:**

Lab_11 > ex-3 > event.js > ...

```javascript
const EventEmitter = require("events");

const emitter = new EventEmitter();

// Listener 1
emitter.on("greet", (name) => {
  console.log(`Hello ${name}`);
});

// Listener 2
emitter.on("greet", (name) => {
  console.log(`Welcome ${name}`);
});

// Emit event
emitter.emit("greet", "Karthik");
```

```
PS C:\Users\SYS\Desktop\Web_tecnologies_lab_work> cd Lab_11/ex-3
PS C:\Users\SYS\Desktop\Web_tecnologies_lab_work\Lab_11\ex-3> node event.js
Hello Karthik
Welcome Karthik
PS C:\Users\SYS\Desktop\Web_tecnologies_lab_work\Lab_11\ex-3>
```